

CSE200 • B2 • Group 2

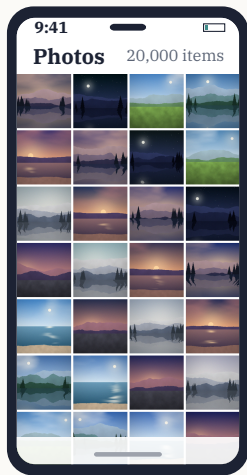
VARIATIONAL AUTOENCODER

Presented by

Mahir Mahtab • Nusaiba Nehleen Bhuiyan • Ramisa Musarrat Sujana

Department of CSE, BUET

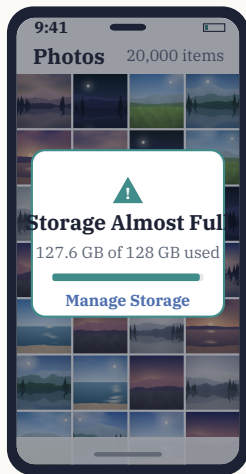
The Problem: Storage Almost Full



Imagine your phone has

20,000 photos

The Problem: Storage Almost Full



Storage Almost **Full!**

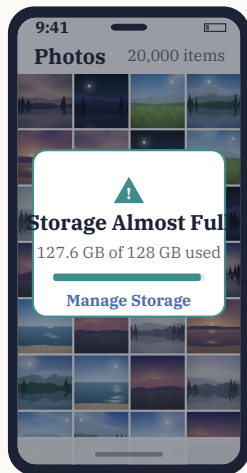
Storage

127.6 GB of 128 GB used



■ Photos 112 GB ■ Apps 10 GB ■ System 5.6 GB

The Problem: Storage Almost Full



Storage Almost **Full!**

Storage 127.6 GB used **only 0.4 GB free**



■ Photos 112 GB ■ Apps 10 GB ■ System 5.6 GB

90% Storage

Occupied by Photos!

What do you do?

What do you do?



Delete Photos

not ideal: once gone, they're gone forever

What do you do?



Delete Photos

not ideal: once gone, they're gone forever



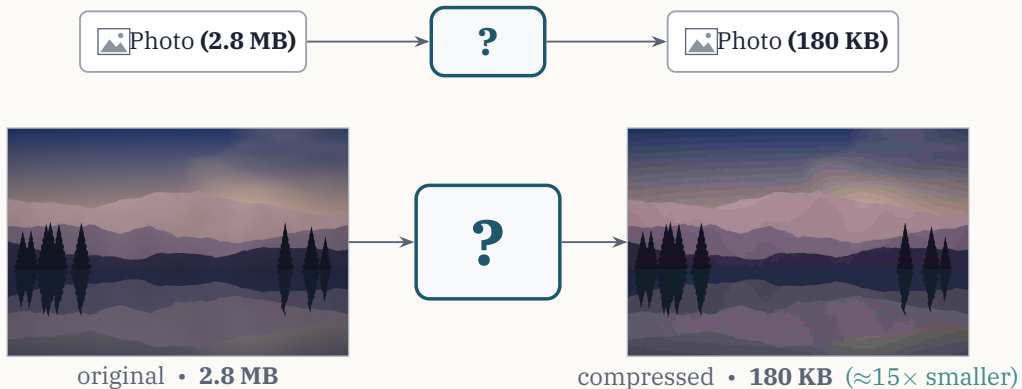
A Smarter Solution

keep every photo, but use lesser space

Compression Saves Storage



Compression Saves Storage



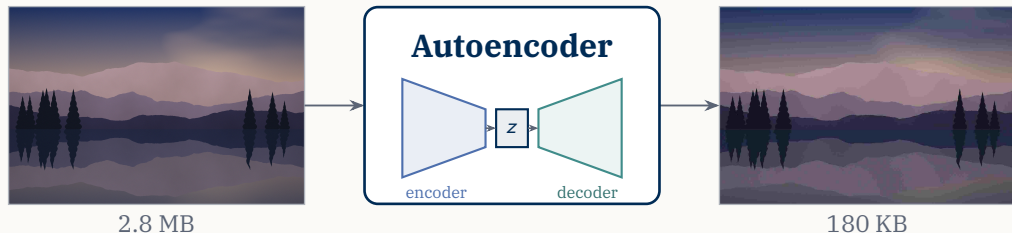
Compressed image is **15 times smaller**

Enter the Autoencoder



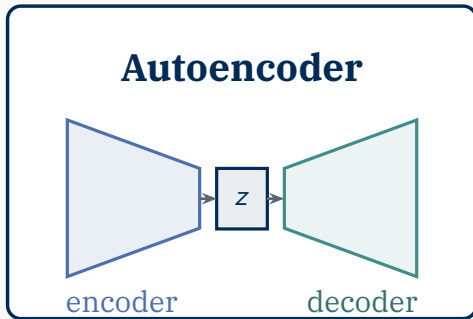
What goes inside the **mysterious middle box**?

Enter the Autoencoder



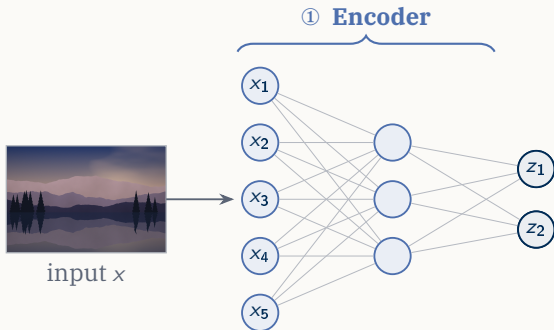
This box is exactly where **autoencoders** come in.

Enter the Autoencoder



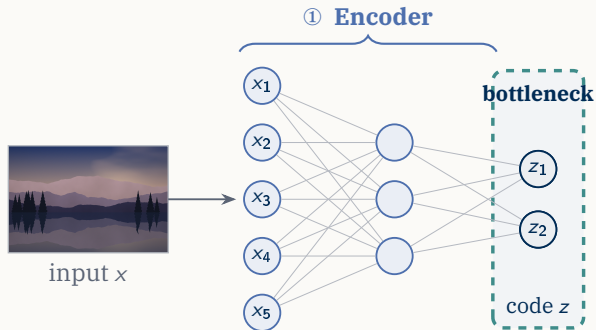
A neural network that learns to **compress** data and **reconstruct** it
It has two main parts: **1. Encoder** **2. Decoder**

Autoencoder Architecture



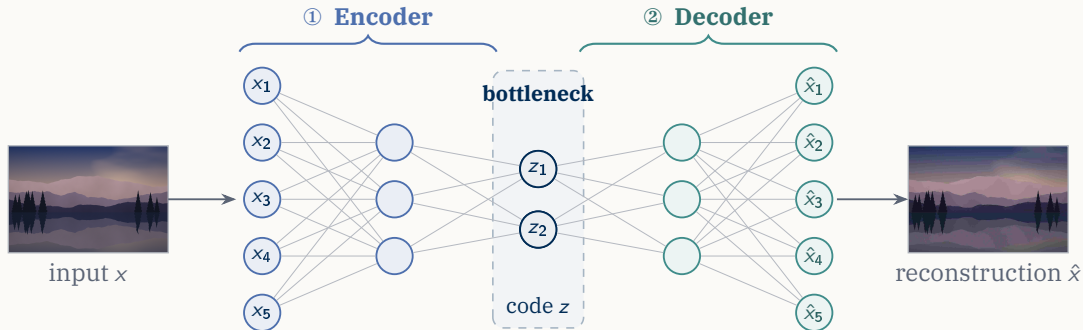
The **encoder** compresses the image, layer by layer, until it reaches the bottleneck.

Autoencoder Architecture



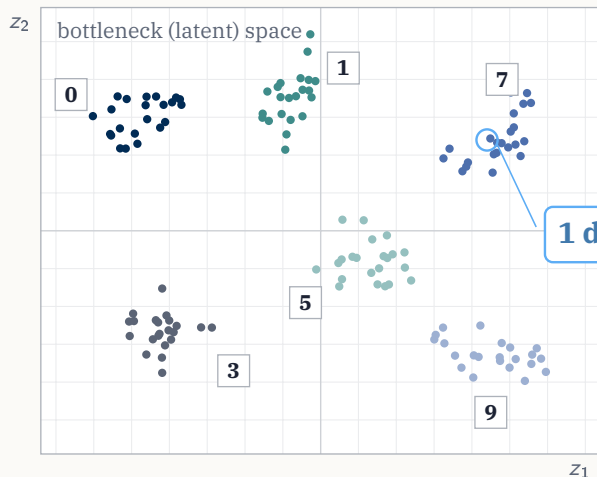
The **bottleneck** allows only the *essential* information to pass which is called *latent code*, represented in the *latent space*

Autoencoder Architecture



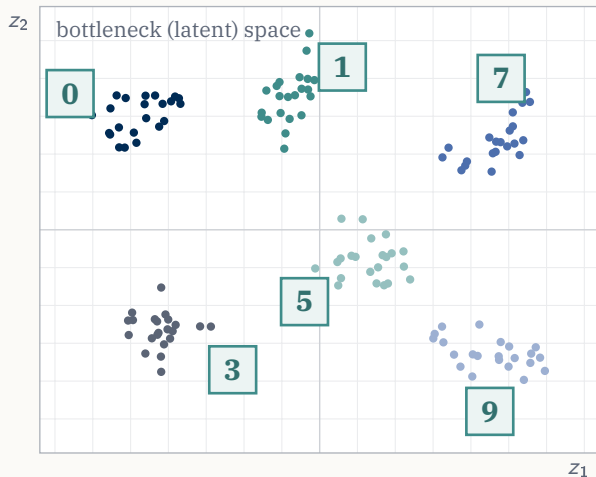
The **decoder** uses that latent code to rebuild the image: $\hat{x} \approx x$.

The Problem with Autoencoders



Here in this bottleneck space, each point is exactly **one image** after compression

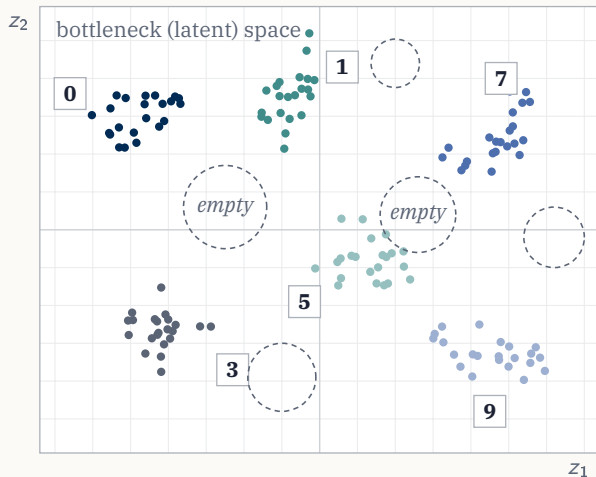
The Problem with Autoencoders



Here in this bottleneck space, each point is exactly **one image** after compression

Notice how similar images form separate **islands**, such as 0, 1, 5, 7, and 9

The Problem with Autoencoders

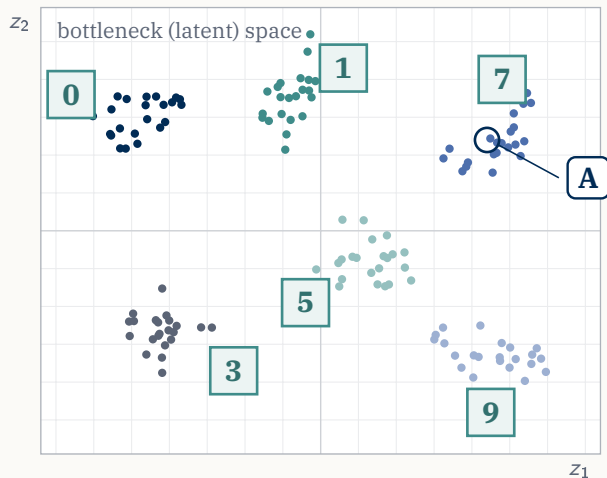


Here in this bottleneck space, each point is exactly **one image** after compression

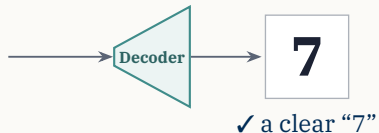
Notice how similar images form separate **islands**, such as 0, 1, 5, 7, and 9

There are lots of **empty spaces** between them. The decoder has not seen training examples from these regions.

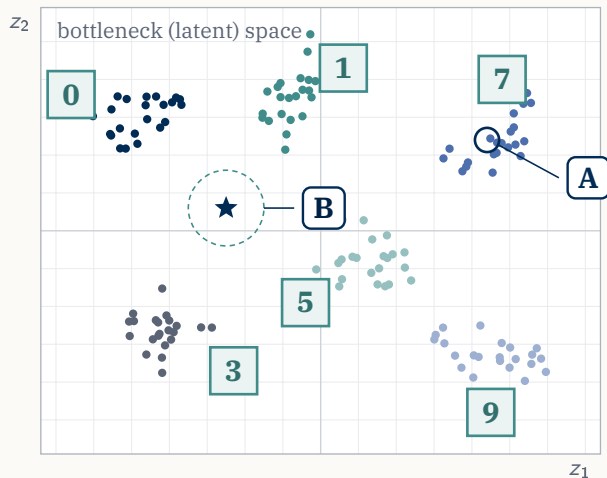
What Lives in the Gaps?



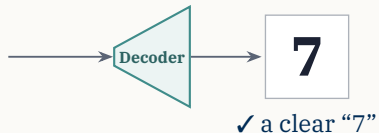
A point inside an island



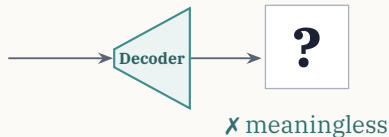
What Lives in the Gaps?



A point inside an island



B random point from an empty gap



The Key Insight

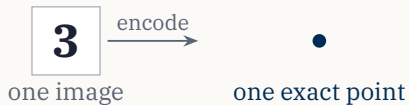
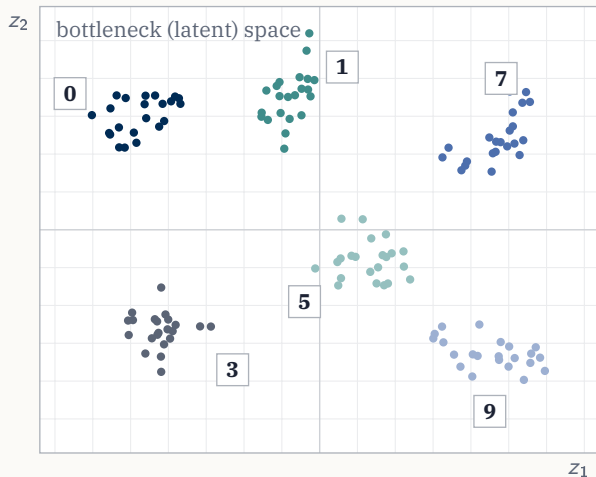
Instead of storing each image as one exact point, what if every image were represented as a **distribution**?

The Key Insight

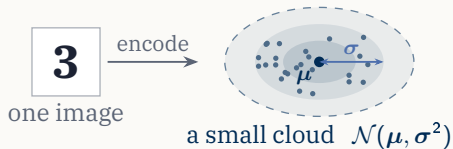
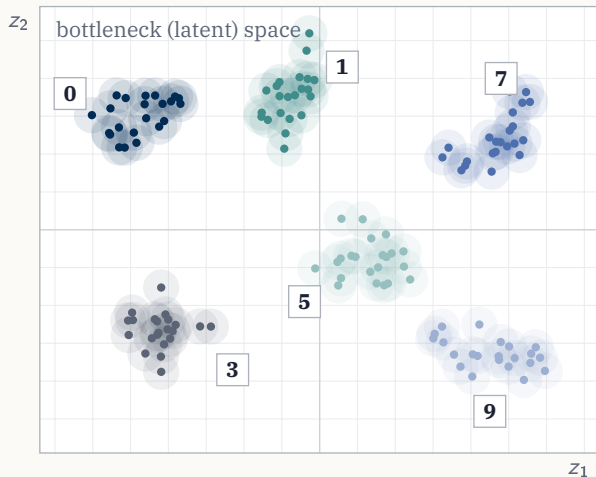
Instead of storing each image as one exact point, what if every image were represented as a **distribution**?

That's the core idea behind the Variational Autoencoder (VAE)

From Points to Distributions



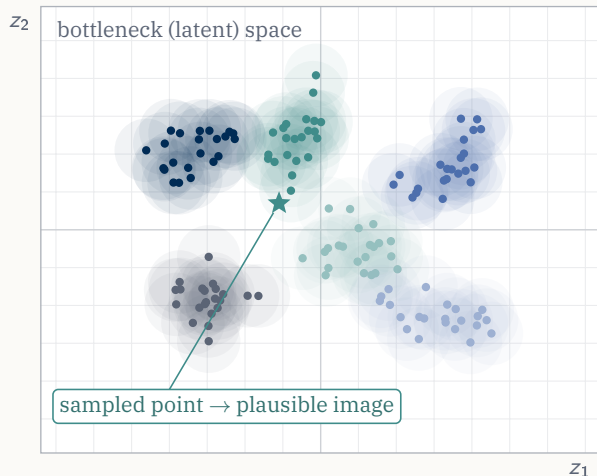
From Points to Distributions



VAE: one image $\rightarrow$ one **distribution**

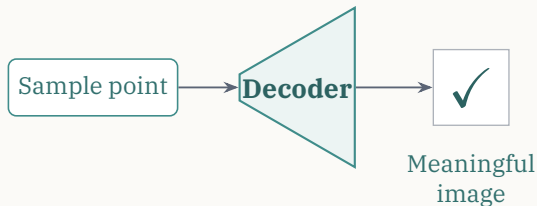
Each image now occupies a **small area**.

From Points to Distributions

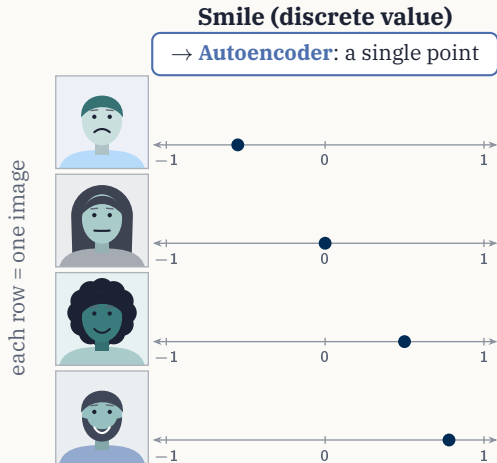


Notice, the distributions overlap, which **reduces empty gaps**.

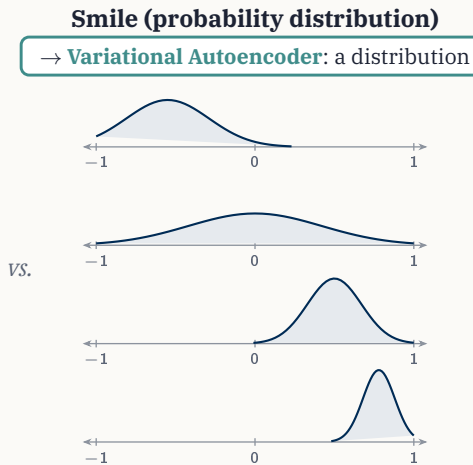
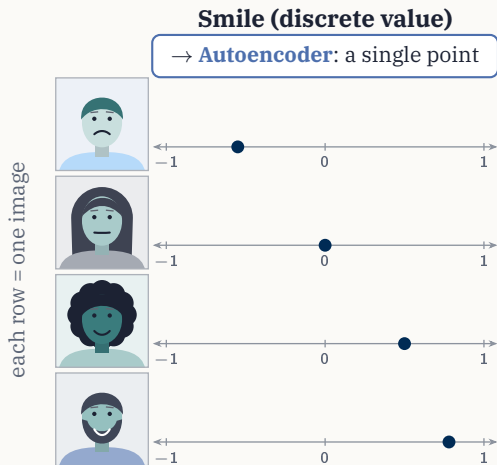
As a result, we can choose a new point from that area and it will still decode into a meaningful image.



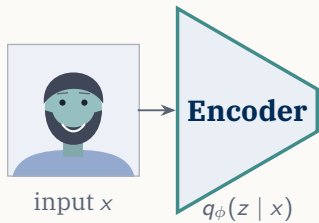
One Image, One Distribution



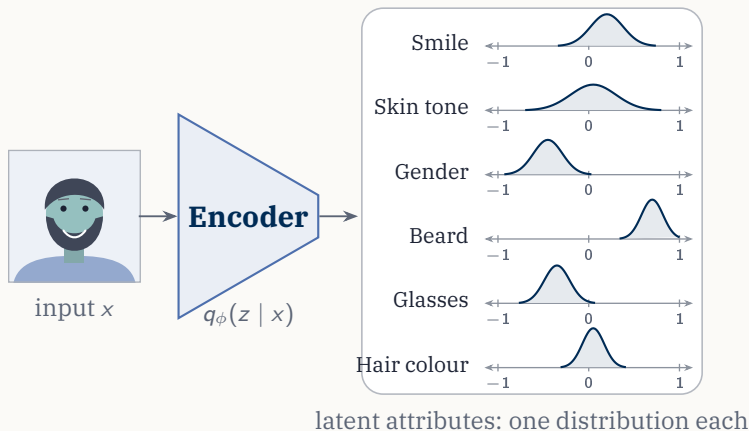
One Image, One Distribution



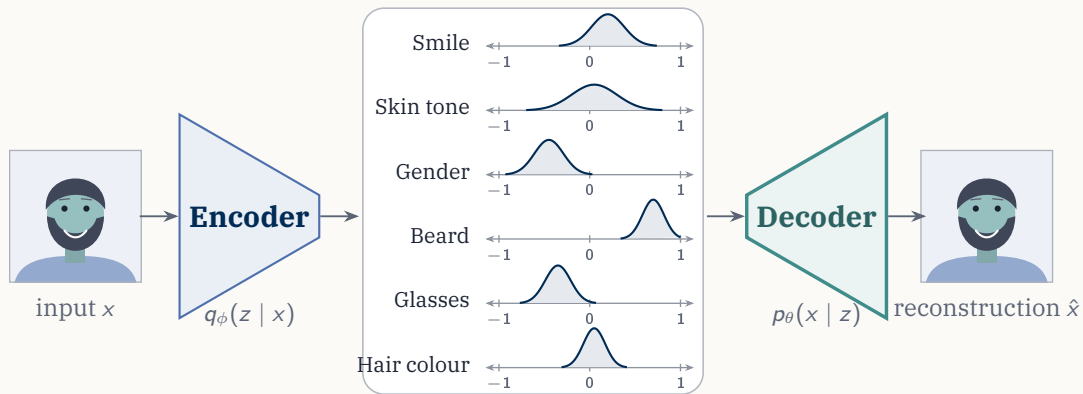
VAE Simplified: Latent Attributes



VAE Simplified: Latent Attributes

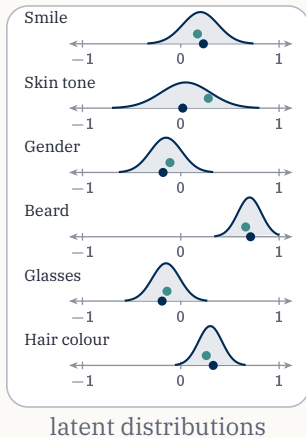


VAE Simplified: Latent Attributes

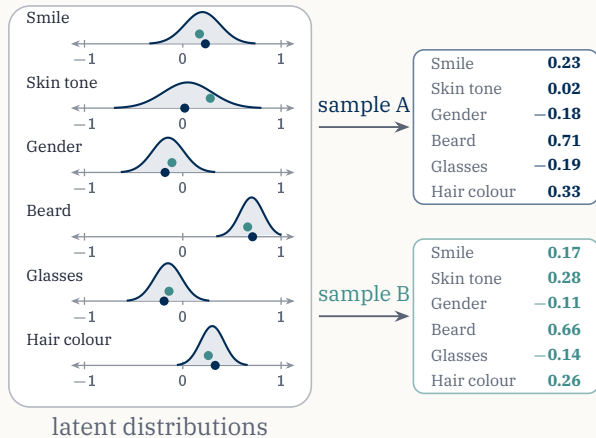


latent attributes: one distribution each

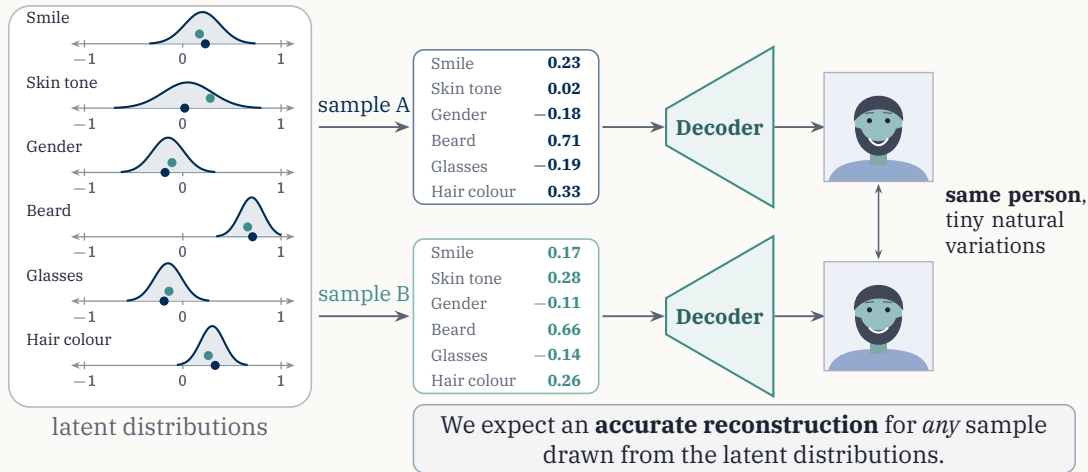
VAE Simplified: Sampling and Decoding



VAE Simplified: Sampling and Decoding



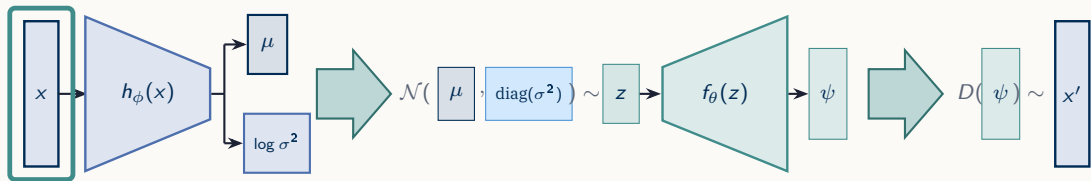
VAE Simplified: Sampling and Decoding



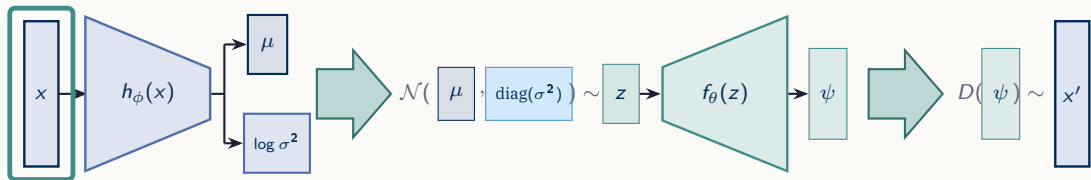
Back to the 80's



Input

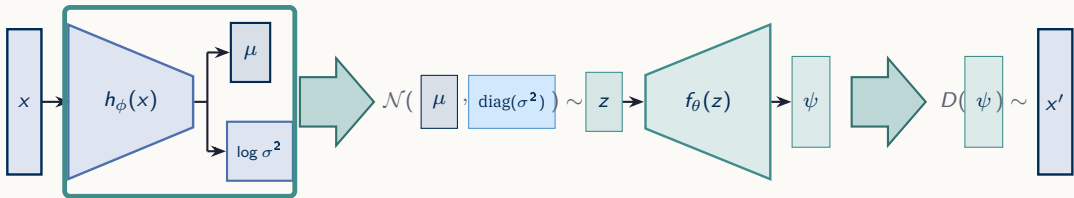


Input

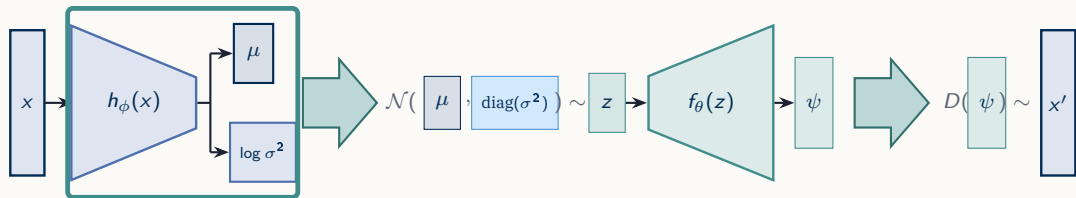


The **Variational Autoencoder** begins with an input sample x , which can be an **image**, **text**, or **any data**.

Encoder

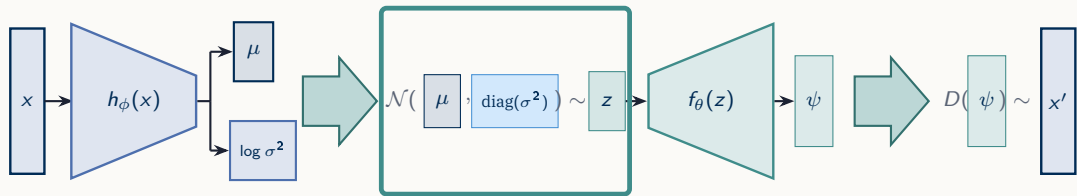


Encoder



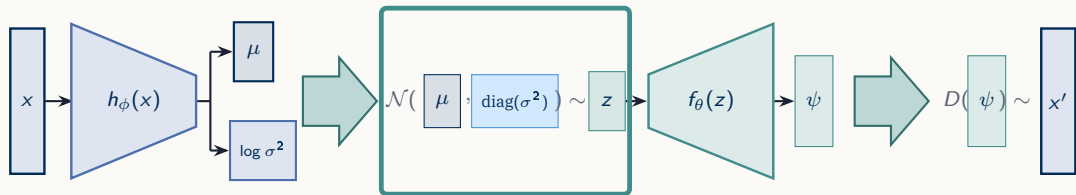
- The **Encoder** maps the input x into a **Latent Space Representation**.
- Outputs:
 - **Mean** (μ)
 - **Log Variance** ($\log \sigma^2$)

Latent Variable



z represents a compressed **probabilistic latent space**.

Latent Variable

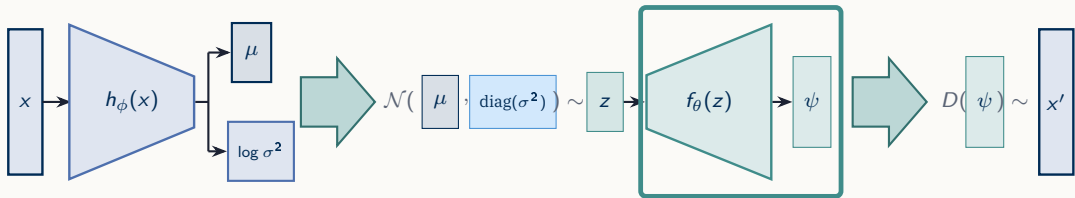


z represents a compressed **probabilistic latent space**.

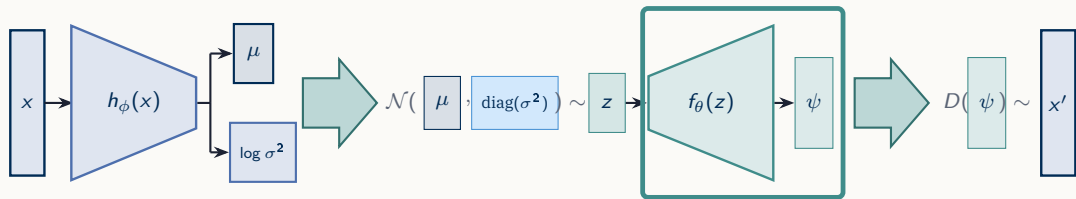
Instead of fixed values, z is drawn from a distribution defined by the encoder:

$$p(z | x) \sim \mathcal{N}(\mu, \sigma^2)$$

Decoder

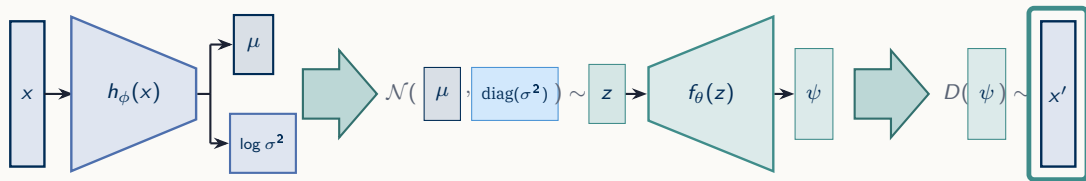


Decoder

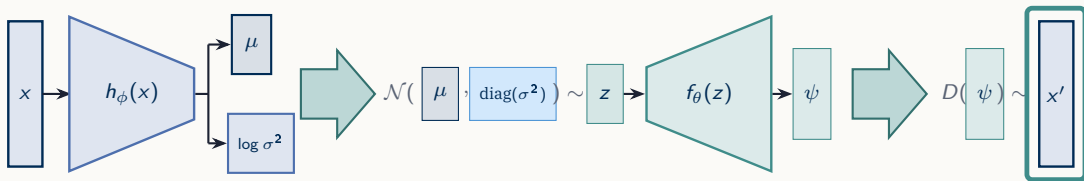


The decoder maps z to reconstructed data x' via $p_\theta(x | z)$.

Output: x'



Output: x'



The **VAE** generates data x' with the same input dimension as x using the latent variable z , i.e., $x' \in \mathbb{R}^d$ where d is the input dimension.

Reconstruction Loss

Reconstruction Loss: Measures the difference between the input x and the reconstructed output $\hat{x}$.

Reconstruction Loss

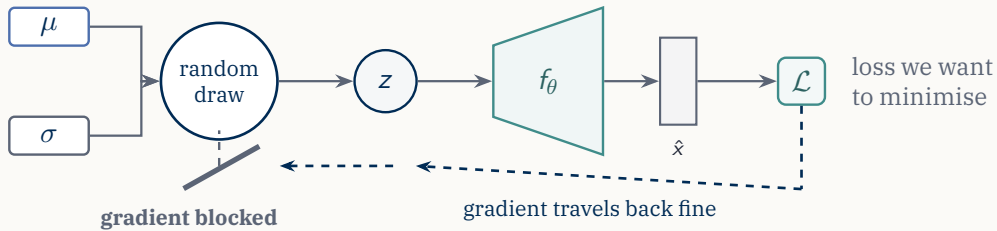
Reconstruction Loss: Measures the difference between the input x and the reconstructed output $\hat{x}$.

Mean Squared Error (MSE): measures the squared difference between x and $\hat{x}$.

$$\mathcal{L}_{\text{recon}} = \|x - \hat{x}\|^2$$

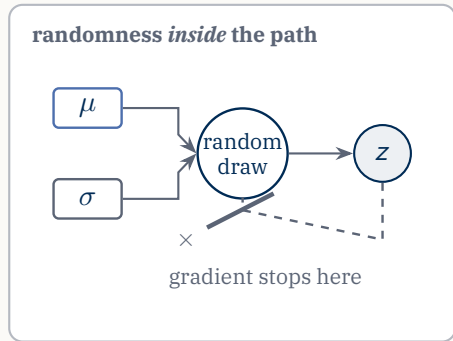
Binary Cross-Entropy (BCE): commonly used when the data is binary or normalized like pixel probabilities.

Why Sampling Breaks Training



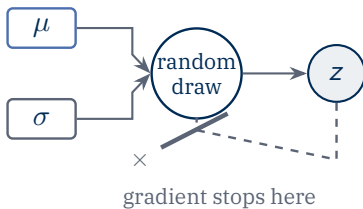
The loss gradient travels backward through the decoder, but **direct random sampling blocks it from reaching μ and σ .**

The Reparameterization Trick

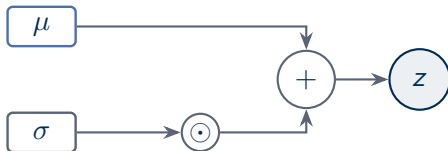


The Reparameterization Trick

randomness *inside* the path

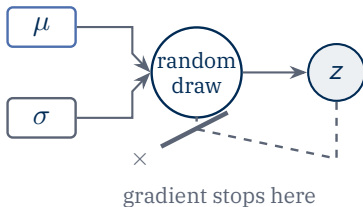


randomness moved *outside*

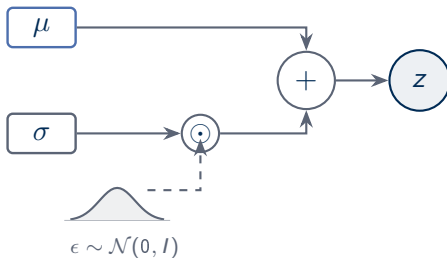


The Reparameterization Trick

randomness *inside* the path

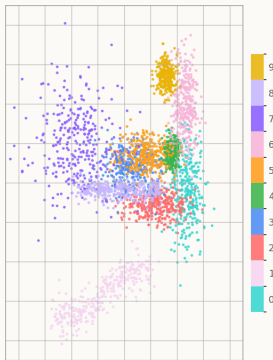


randomness moved *outside*



Reparameterize: $z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$

Only Reconstruction Loss



Problem: With only **reconstruction loss**, the model may reconstruct data well, but the latent space can become **irregular, discontinuous, and hard to sample from**. Random points may fall into weak regions, so generation becomes unreliable.

Kullback-Leibler (KL) Divergence

Measures the divergence between $q_\phi(z|x)$ and the prior $p(z)$, typically $\mathcal{N}(0, I)$.

Kullback-Leibler (KL) Divergence

Measures the divergence between $q_\phi(z|x)$ and the prior $p(z)$, typically $\mathcal{N}(0, I)$.

Regularization Purpose

Promotes a smooth, continuous latent space, fostering:

- **Continuity**
- **Completeness**

Kullback-Leibler (KL) Divergence

Measures the divergence between $q_\phi(z|x)$ and the prior $p(z)$, typically $\mathcal{N}(0, I)$.

Regularization Purpose

Promotes a smooth, continuous latent space, fostering:

- **Continuity**
- **Completeness**

KL Divergence Formula

$$\mathcal{L}_{KL} = D_{KL}(q_\phi(z|x) \parallel p(z)) = \int q_\phi(z|x) \log \frac{q_\phi(z|x)}{p(z)} dz$$

Evidence Lower Bound (ELBO)

ELBO: Overview

ELBO balances two objectives: **Reconstruction Quality** and **Latent Space Regularization**.

Evidence Lower Bound (ELBO)

ELBO: Overview

ELBO balances two objectives: **Reconstruction Quality** and **Latent Space Regularization**.

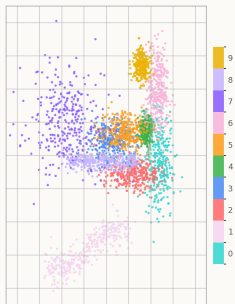
Goal

Maximizing ELBO is equivalent to minimizing the VAE loss:

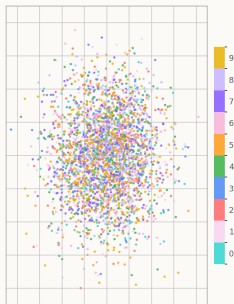
$$\mathcal{L}_{\text{VAE}} = \mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{KL}}$$

ELBO : Impact Visualization

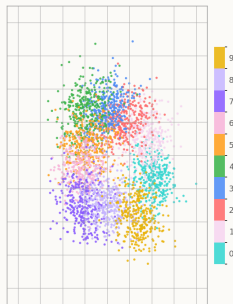
Only reconstruction loss



Only KL divergence



Reconstruction loss and KL divergence

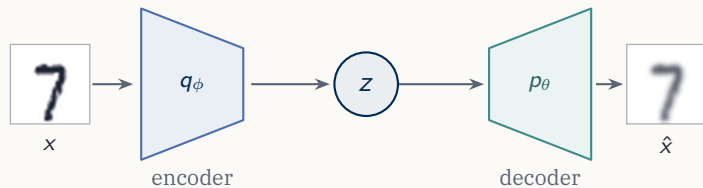


ELBO combines both goals: $\text{ELBO} = \mathbb{E}_{q_{\phi}(z|x)}[\log p_{\theta}(x | z)] - D_{KL}(q_{\phi}(z | x) \| p(z))$

In practice: **reconstruct well** and **keep the latent space regular**.

Generating New Data

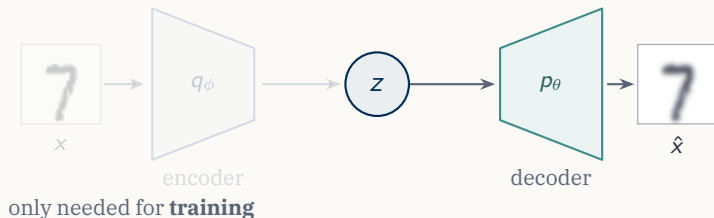
Training is over. To make *new* images, we only need half of the network.



While training, all three stages run: **encode**, **sample**, **decode**.

Generating New Data

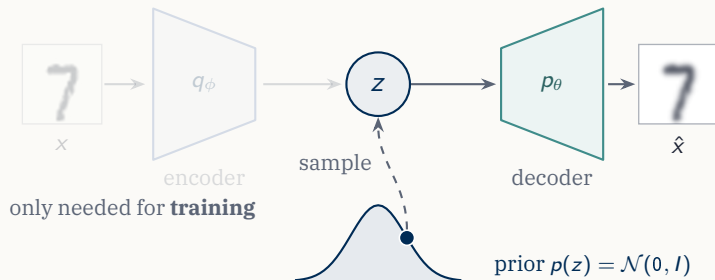
Training is over. To make *new* images, we only need half of the network.



To generate there is no input image to encode, so take z **straight from the prior**.

Generating New Data

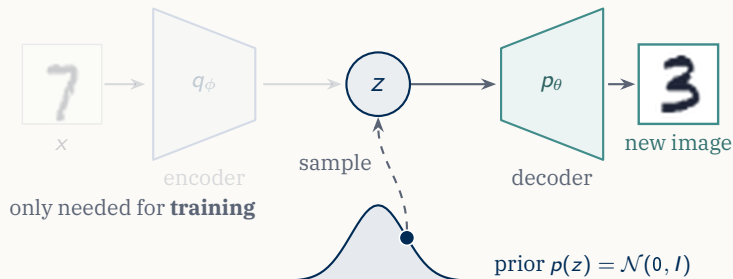
Training is over. To make *new* images, we only need half of the network.



To generate there is no input image to encode, so take z **straight from the prior**.

Generating New Data

Training is over. To make *new* images, we only need half of the network.



Every new draw gives a **different** digit:

$$z^{(1)} \sim p(z) \longrightarrow \boxed{3}$$

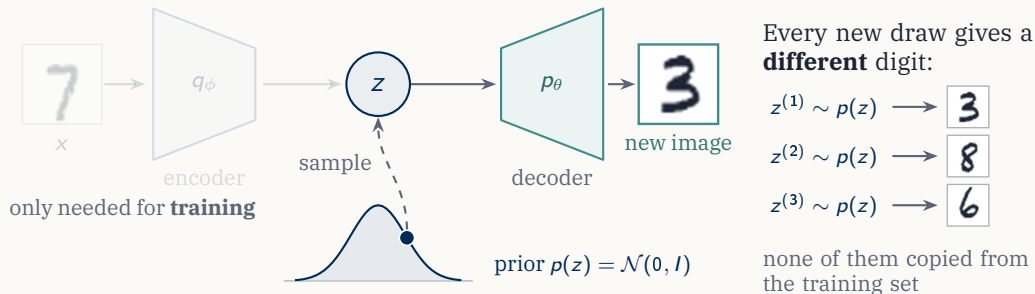
$$z^{(2)} \sim p(z) \longrightarrow \boxed{8}$$

$$z^{(3)} \sim p(z) \longrightarrow \boxed{6}$$

none of them copied from the training set

Generating New Data

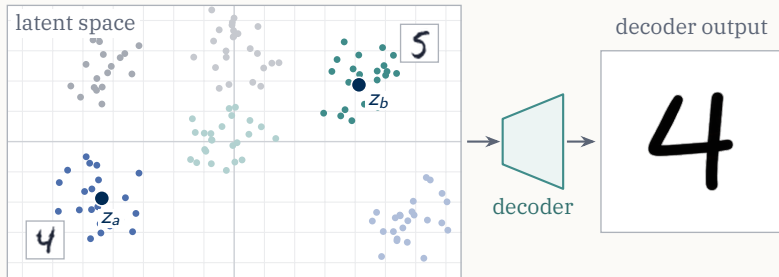
Training is over. To make *new* images, we only need half of the network.



The KL term trained every encoding to sit inside $\mathcal{N}(0, I)$, so a random draw lands where the decoder already knows what to draw. **Learning to generate, not just compress.**

Walking Through Latent Space

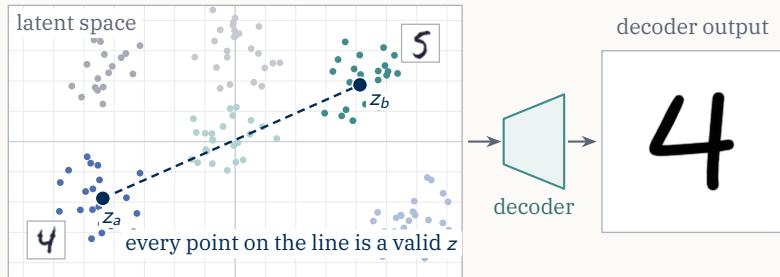
One sample gives one image. What if we don't jump, but **walk**?



At z_a , the decoder draws a **4**.

Walking Through Latent Space

One sample gives one image. What if we don't jump, but **walk**?

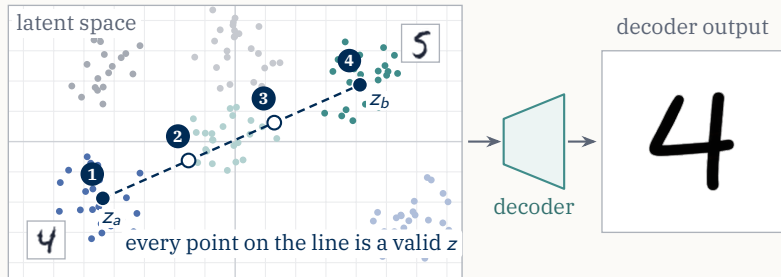


At z_a , the decoder draws a **4**.

Now slide z along the line to $z_b \dots$

Walking Through Latent Space

One sample gives one image. What if we don't jump, but **walk**?



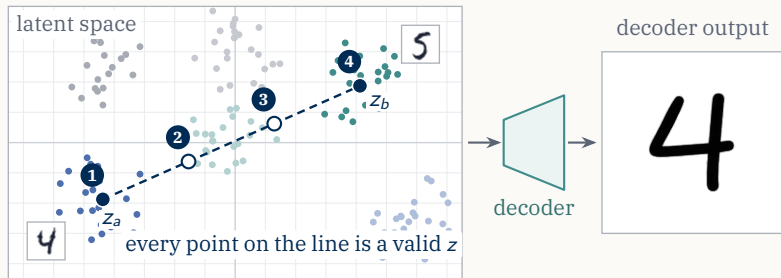
At z_a , the decoder draws a 4.

Now slide z along the line to z_b ...



Walking Through Latent Space

One sample gives one image. What if we don't jump, but **walk**?

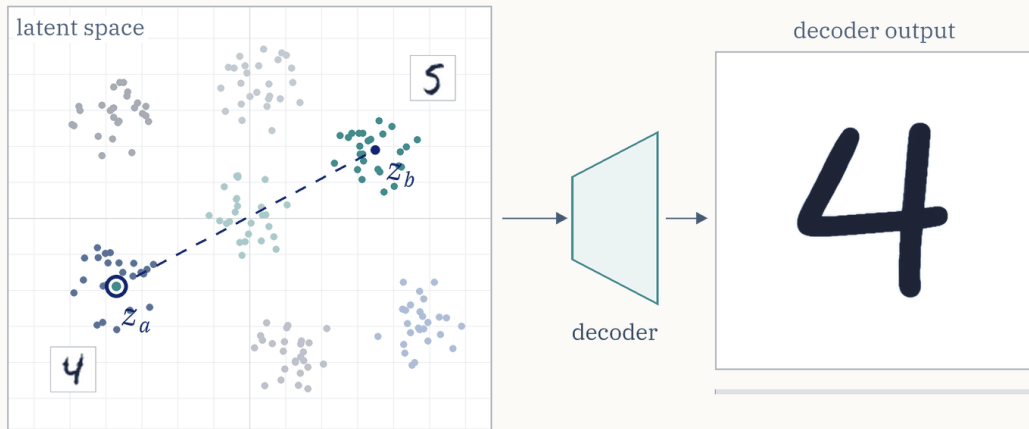


Every step
decodes to a
real-looking
digit.

Remember the
gaps from Part 1?
The KL term
filled them.



Walking Through Latent Space



Conditional VAE: Choosing What Comes Out

✗ **Vanilla VAE: we can't pick the digit**



Conditional VAE: Choosing What Comes Out

✗ Vanilla VAE: we can't pick the digit



✓ Conditional VAE (CVAE)

give the label y to **both** halves

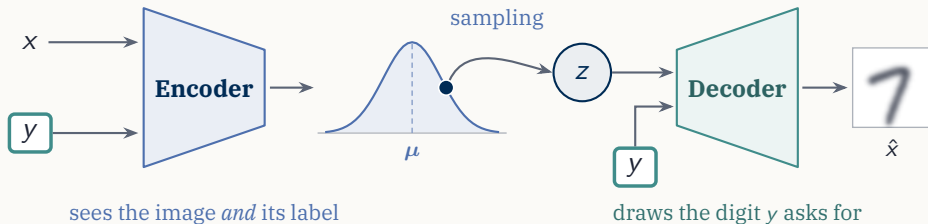
Conditional VAE: Choosing What Comes Out

✗ Vanilla VAE: we can't pick the digit



✓ Conditional VAE (CVAE)

give the label y to **both** halves



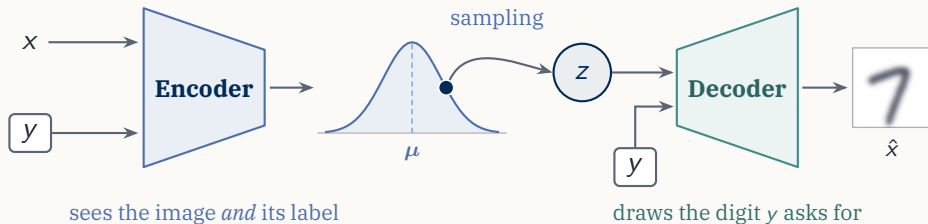
Conditional VAE: Choosing What Comes Out

✗ Vanilla VAE: we can't pick the digit



✓ Conditional VAE (CVAE)

give the label y to **both** halves



sees the image *and* its label

draws the digit y asks for

The label y says **which** digit.
So z only stores **how** it is written.

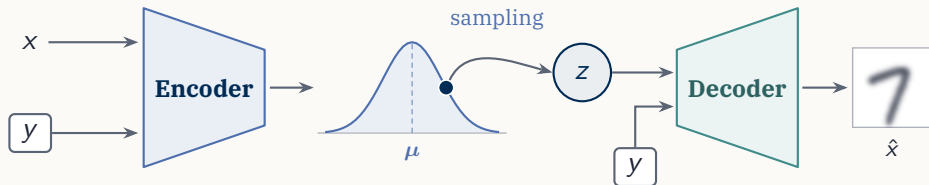
Conditional VAE: Choosing What Comes Out

✗ Vanilla VAE: we can't pick the digit



✓ Conditional VAE (CVAE)

give the label y to **both** halves



sees the image *and* its label

draws the digit y asks for

The label y says **which** digit.
So z only stores **how** it is written.

$y = 7$: **always a 7**,
only the *style* varies.



Where VAEs Are Used



Anomaly Detection

bad reconstruction = something unusual



Drug & Molecule Design

sample z to get new molecules

Where VAEs Are Used



Anomaly Detection

bad reconstruction = something unusual



Drug & Molecule Design

sample z to get new molecules



Recommendation

a latent “taste” for every user

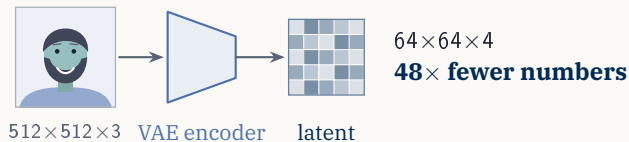


Data Augmentation

decode extra training examples

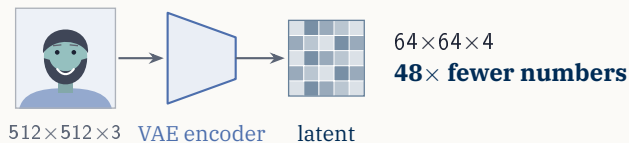
How Diffusion Models Use a VAE

1 Compress

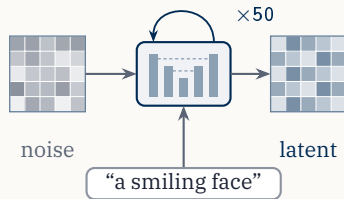


How Diffusion Models Use a VAE

1 Compress

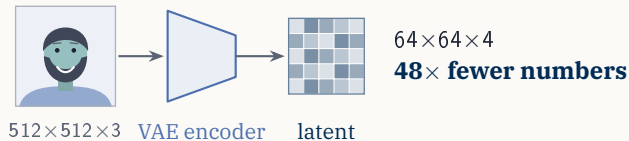


2 Generate

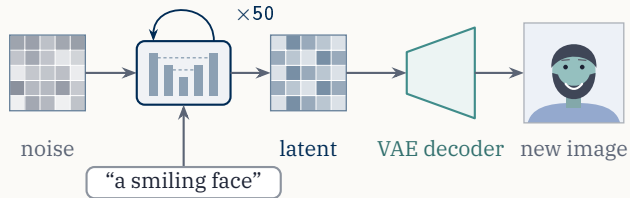


How Diffusion Models Use a VAE

1 Compress



2 Generate

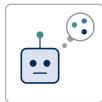


What's Next for VAEs



Video generation

Compressing space *and* time, so AI can make whole clips.



Robots that imagine

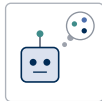
Practising inside a learned latent world before acting.

What's Next for VAEs



Video generation

Compressing space *and* time, so AI can make whole clips.



Robots that imagine

Practising inside a learned latent world before acting.



New materials

Sampling new crystals for better batteries and solar cells.



Reading the brain

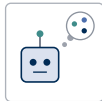
Turning brain scans back into the images a person saw.

What's Next for VAEs



Video generation

Compressing space *and* time, so AI can make whole clips.



Robots that imagine

Practising inside a learned latent world before acting.



New materials

Sampling new crystals for better batteries and solar cells.



Reading the brain

Turning brain scans back into the images a person saw.

compress → **smooth** → **sample**: the same recipe, aimed at new problems.

Thank You